

Analyze your resume through the eyes of an ATS bot, a recruiter, and a hiring manager — all at once. Get actionable AI suggestions, predict interview questions, and benchmark against top candidates in your field.

Phase 1 Weeks 1–8 Foundation & MVP Auth, parsing, ATS scoring, persona analysis, billing, launch	Phase 2 Weeks 9–16 Unique Differentiators JD adapter, A/B tester, interview predictor, benchmarking
Phase 3 Weeks 17–24 Career Intelligence Trajectory AI, salary calibration, GitHub sync, B2B	Phase 4 Month 7+ Advanced V2 Multilingual adapter, bias scanner, white-label, enterprise SSO

105 Total tasks	24+ Weeks to full launch	9 Unique differentiators	4 Pricing tiers
---------------------------	------------------------------------	------------------------------------	---------------------------

What is ResumeAI?

ResumeAI is a full-stack SaaS application that goes far beyond keyword matching. Instead of just checking if your resume contains the right words, it simulates three different reviewer perspectives, measures your language confidence, predicts what a recruiter will ask you in an interview, and compares your resume against top-performing candidates in your exact role and industry.

The core insight: Most resume tools tell you *what's missing*. ResumeAI tells you *why you're being filtered out* and exactly how to fix it.

How we compare to existing tools

Competitor	Their gap	Our difference
Jobscan	Keyword match vs JD only	Persona analysis + tone scoring + interview predictor
Resume Worded	Generic tips, no benchmarking	Role-specific benchmarking vs real top-10% candidates
Rezi.ai	ATS scoring only	Full career intelligence — trajectory, salary, interviews
Enhancv	Design-focused, shallow AI	Deep content quality analysis, not just layout
Teal HQ	Job tracker, AI is secondary	Resume AI as the primary product, not an afterthought

Competitor	Their gap	Our difference
Zety/Novoresume	Template builder, basic suggestions	ATS simulation + authentic voice preservation

Key Features & Differentiators

Core analysis (available on all plans)

ATS compatibility scoring — Parse and score resume against applicant tracking system rules, flagging format issues

Section-by-section grading — Individual scores for summary, experience, skills, and education with targeted fixes

Bullet point rewriter — Transform passive bullets into action-verb + metric + outcome format via Claude API

Format & structure audit — Flag fonts, column layouts, tables, and elements that break ATS parsing

Unique differentiators (Pro & Career plans)

Persona-based analysis	Views resume as ATS bot, HR recruiter, and hiring manager simultaneously. Shows exactly where each reviewer stops reading.
Tone & confidence analyzer	Detects passive language, hedging phrases, filler words. Gives confidence score per section with AI rewrites.
Interview question predictor	Generates likely recruiter questions from vague or weak resume spots. Lets you prepare before the call.
JD adapter	Paste any job description to get a tailored resume variant optimized for that specific role in one click.
Resume A/B tester	Compare two resume versions head-to-head for a specific JD. Data-driven selection, not guesswork.
Skill authenticity checker	Cross-references claimed skills against project descriptions. Flags skills listed without evidence.
Industry benchmark engine	Compares your resume against anonymized top-10% resumes in your exact role and industry.
Career gap advisor	Detects employment gaps and suggests honest reframing strategies to turn them into a narrative.
Career trajectory predictor	Predicts realistic next roles and skill gaps using live market data from job boards.

V2 advanced features (Career+ plans)

- Real-time salary calibration based on resume strength + role + location
- GitHub / Dribbble / portfolio integration to link projects to resume bullets
- Multilingual + cultural format adapter (US, German CV, Japanese rirekisho formats)
- Language bias and inclusivity scanner — flag gendered, age-signalling language
- White-label licensing for bootcamps and staffing agencies
- Enterprise SSO via SAML 2.0 + bulk screening API

Tech Stack

Frontend	Next.js 14 (App Router), TypeScript, Tailwind CSS, shadcn/ui, Zustand, React Query (TanStack), Framer Motion
Backend	FastAPI (Python 3.11), Celery, Redis, Pydantic v2, Uvicorn / Gunicorn

AI / ML	Anthropic Claude API (primary), OpenAI GPT-4o (fallback), LangChain / LangGraph, Sentence-Transformers, spaCy, NLTK
File Parsing	PyMuPDF / pdfplumber (PDF), python-docx (DOCX), unstructured.io (fallback), Tesseract OCR (scanned PDFs)
Database	PostgreSQL (primary), Prisma ORM, pgvector / Pinecone (embeddings), Redis (cache + sessions)
Auth & Pay	Clerk (auth + Google/LinkedIn OAuth), Stripe (subscriptions + usage metering)
Storage	AWS S3 (resume files + exports), AWS EC2 (backend), AWS RDS (production DB)
DevOps	Docker + Compose (local dev), Vercel (frontend), GitHub Actions (CI/CD), Sentry (errors), PostHog (analytics)

Project Structure

```

resume-analyzer/
  frontend/
    Next.js 14 app
    app/
      (auth)/      Login, signup, OAuth callbacks
      dashboard/   User home, resume history
      analyze/
        upload/    Drag-drop upload screen
        result/[id]/ Full analysis result page
        compare/   A/B tester screen
      jd-adapter/   JD paste -> tailored resume
      career-path/  Trajectory predictor
      interview-prep/ Predicted questions page
      salary/       Salary calibration (V2)
      billing/      Stripe plan management
    components/
      analysis/     ScoreCard, PersonaView, ToneChart
      resume/       ResumeEditor, BulletRewriter
      benchmark/    IndustryCompare charts
      shared/       Button, Badge, Modal, Toast
    lib/            API client, hooks, utilities
    store/          Zustand state slices

  backend/
    FastAPI Python service
    app/
      routers/      resume.py, analysis.py, jd.py,
                    career.py, benchmark.py, salary.py
      models/        SQLAlchemy DB models
      schemas/       Pydantic request/response schemas
      services/      parser_service, storage_service,
                    stripe_service
      middleware/    Auth, rate limiting, CORS
      db/            Alembic migrations, DB session

  ai-engine/
    All LLM logic (importable module)
    analyzers/
      ats_analyzer, persona_analyzer,
      tone_analyzer, skill_checker,
      gap_disguiser, bias_scanner (V2)
    generators/
      bullet_rewriter, jd_adapter,
      interview_predictor, career_predictor,
      salary_estimator (V2), cultural_adapter (V2)
    comparators/
      ab_tester, benchmark_engine
    prompts/
      Versioned prompt templates

```

```

embeddings/      Vectorize resume sections
pipeline.py      Orchestrates full analysis run

workers/         Celery async task definitions
analyze_task.py  Full AI pipeline (async)
benchmark_task.py Industry comparison job
github_sync_task.py Portfolio pull (V2)
celery_app.py    Celery + Redis config

infra/           Docker, CI/CD, AWS configs
docker-compose.yml
.env.example
README.md

```

Phase-by-Phase Work Plan

PHASE 1 — FOUNDATION & MVP

Weeks 1–8 | 33 tasks | Launch-ready at end

Week	Task	Layer	Description
1–2	Initialise monorepo	infra	Folder structure, git init, .gitignore, .env.example
1–2	Next.js 14 setup	frontend	App router, TypeScript, Tailwind, shadcn/ui init
1–2	FastAPI boilerplate	backend	main.py, routers/, CORS, health check endpoint
1–2	Docker Compose setup	infra	Services: frontend, backend, postgres, redis, worker
1–2	PostgreSQL + Alembic	database	DB connection, migrations runner, base models
1–2	Celery + Redis worker	backend	celery_app.py, test task, worker container
3–4	Clerk auth integration	frontend	Sign up, login, Google OAuth, session middleware
3–4	JWT verification middleware	backend	FastAPI dependency that validates Clerk JWT on every request
3–4	Resume upload endpoint	backend	POST /resume/upload, multipart form, S3 storage
3–4	PDF parser	backend	pdfplumber extraction, section detection, JSON output
3–4	DOCX parser	backend	python-docx extraction, same JSON schema as PDF parser
3–4	Parser unit tests	backend	Test against 10+ real resume samples across formats
3–4	Upload UI	frontend	Drag-drop component, file type validation, upload progress
5–6	Claude API integration	AI	anthropic SDK setup, base LLM wrapper, error handling
5–6	ATS scorer	AI	Keyword density, format check, parseable structure score
5–6	Section grader	AI	Individual scores for summary, experience, skills, education
5–6	Bullet rewriter	AI	Action verb + metric + outcome rewrite prompt + JSON output
5–6	Tone & confidence analyzer	AI	spaCy passive detection + Claude confidence score per section
5–6	Persona analyzer	AI	3 separate Claude calls: ATS bot, HR recruiter, hiring manager
5–6	Async pipeline	backend	Celery task wiring all analyzers, status polling endpoint
5–6	Prompt version store	AI	ai-engine/prompts/ versioned templates, no inline prompts
7–8	Overview results page	frontend	Score card, section grades, top 3 quick wins display
7–8	Persona view tabs	frontend	ATS / HR / HM tab UI with drop-off indicators
7–8	Inline bullet rewriter UI	frontend	Click bullet -> AI suggestion -> accept/reject interaction
7–8	User dashboard	frontend	Resume history, score over time chart, quick actions
7–8	Stripe billing setup	backend	Free + Pro plans, checkout flow, webhook handler

Week	Task	Layer	Description
7–8	Feature gating	backend	Middleware to lock persona/tone behind Pro plan
7–8	Landing page	frontend	Hero, live demo embed, pricing table, social proof
7–8	Deploy to production	infra	Vercel (frontend) + EC2 + RDS + S3 production setup
7–8	Sentry + PostHog	infra	Error tracking and product analytics integration

PHASE 2 — UNIQUE DIFFERENTIATORS

Weeks 9–16 | 28 tasks | Virality hooks at end

Week	Task	Layer	Description
9–10	JD adapter AI module	AI	jd_adapter.py — resume JSON + JD text -> tailored resume JSON
9–10	JD adapter endpoint	backend	POST /jd/adapt, async Celery task, result polling
9–10	JD adapter UI	frontend	JD paste box, loading state, side-by-side original vs tailored
9–10	Interview predictor AI	AI	interview_predictor.py — weak bullets -> probing questions
9–10	Interview predictor UI	frontend	Questions panel grouped by resume section, prep tips
9–10	Interview questions DB table	database	Persist questions per analysis_id for later review
11–12	A/B tester AI module	AI	ab_tester.py — score two resumes on same criteria, pick winner
11–12	A/B tester endpoint	backend	POST /analysis/compare, accept two resume IDs + optional JD
11–12	A/B tester UI	frontend	Dual upload, score comparison bars, winner badge + reasoning
11–12	Skill authenticity checker	AI	skill_checker.py — skills list vs experience bullets cross-ref
11–12	Skill audit UI	frontend	Heatmap of evidenced vs unsupported skills, fix suggestions
13–14	Resume embedding pipeline	AI	Sentence-Transformers vectorize resume sections, store in Pinecone
13–14	Benchmark data seeding	database	Curate anonymized top-10% resume samples per role/industry
13–14	Benchmark comparison engine	AI	benchmark_engine.py — cosine similarity + gap analysis
13–14	Benchmark UI	frontend	Radar chart: user vs top 10% across 6 dimensions
13–14	Gap disguiser AI module	AI	gap_disguiser.py — detect employment gaps, suggest narratives
13–14	Gap advisor UI	frontend	Timeline view, detected gaps highlighted, reframe strategies panel
15–16	Shareable score card	frontend	Public result page with OG image, LinkedIn share button
15–16	OG image generator	frontend	Satori/Vercel OG to generate score card preview images
15–16	Referral system	backend	Unique referral links, 1 free analysis credit per referral
15–16	Usage metering	backend	Track analyses/month, show 3/3 used banner on free plan
15–16	SEO landing pages	UX	Role-specific pages: resume tips for software engineers etc.
15–16	Career plan tier billing	backend	Add \$39 Career plan to Stripe, gate benchmark + gap features
15–16	Email notifications	backend	Analysis complete email, weekly resume tip, re-engagement

PHASE 3 — CAREER INTELLIGENCE

Weeks 17–24 | 24 tasks | Full career coach at end

Week	Task	Layer	Description
17–18	Career predictor AI	AI	career_predictor.py — infer current level, predict next 2 roles
17–18	Skill gap mapper	AI	Compare resume skills vs requirements of predicted next role
17–18	Market data integration	backend	Scrape/API job boards for role frequency & skill demand data
17–18	Career path UI	frontend	Visual trajectory tree, skill gap progress bars, resource links
17–18	Career data caching	backend	Redis cache market data (TTL 24h), reduce API call costs

Week	Task	Layer	Description
19–20	Salary estimator AI	AI	salary_estimator.py — resume strength + role + location -> range
19–20	Salary data source	backend	Integrate Glassdoor/Levels.fyi API or scrape salary data
19–20	Salary calibration UI	frontend	Range bar, percentile position, how to negotiate tips
19–20	Location input	frontend	City/country selector, adjusts salary range dynamically
21–22	GitHub OAuth	backend	Connect GitHub account, fetch public repo list + READMEs
21–22	Project-to-bullet linker	AI	AI matches GitHub repos to resume bullets, suggests additions
21–22	GitHub sync UI	frontend	Project cards with add to resume buttons, strength ratings
21–22	LinkedIn OAuth	backend	OAuth2 flow, pull profile data to pre-fill resume sections
21–22	GitHub sync Celery task	backend	Background worker, re-sync on schedule, webhook on new push
23–24	Expand benchmark database	database	50+ roles x 10 industries, curate anonymized top samples
23–24	Recruiter dashboard scaffold	frontend	Team plan UI, bulk upload interface, seat management
23–24	Bulk analysis API	backend	POST /bulk/analyze, batch Celery tasks, CSV result export
23–24	Team billing (Stripe)	backend	\$199/mo Team plan, seat-based billing, admin dashboard
23–24	API key management	backend	Generate + revoke API keys for B2B integrations
23–24	Performance tuning	infra	Profile slow endpoints, optimise DB queries, add indexes

PHASE 4 — ADVANCED V2

Month 7+ | 20 tasks | Enterprise-ready at end

Week	Task	Layer	Description
V2	Cultural format rules DB	database	Country-specific norms: US 1-page, German CV, Japanese rirekisho
V2	Cultural adapter AI	AI	cultural_adapter.py — reformat + rewrite for target country norms
V2	Multilingual output	AI	Translate resume content via Claude, preserve formatting
V2	Country selector UI	frontend	Flag picker, before/after adapted resume preview
V2	Bias pattern library	database	Curate gendered words, age signals, ability-normative phrases
V2	Bias scanner AI module	AI	bias_scanner.py — flag + suggest neutral alternatives
V2	Bias report UI	frontend	Highlighted text, category tags (gender/age/ability), rewrites
V2	White-label config system	backend	Custom branding: logo, colors, domain per enterprise client
V2	SSO / SAML integration	backend	Enterprise SSO via SAML 2.0 for large org authentication
V2	Usage analytics dashboard	frontend	Admin view: team usage, top issues, score trends over time
V2	Data export	backend	CSV/JSON export of all analyses for enterprise compliance
V2	Webhooks system	backend	POST to client URL on analysis complete, configurable events
V2	Rate limiting	infra	Per-user and per-IP limits on all AI endpoints via Redis
V2	LLM cost tracking	infra	Log token usage per analysis, alert on cost anomalies
V2	Auto-scaling setup	infra	EC2 auto-scaling group for worker fleet under load
V2	LLM fallback routing	AI	Fallback to GPT-4o if Claude API is rate-limited or down
V2	E2E test suite	infra	Playwright tests covering all critical user flows

API Reference

All endpoints require a Clerk JWT as Bearer token. Swagger UI available at /docs when running locally.

Method	Endpoint	Description
POST	/api/v1/resume/upload	Upload and parse a resume file (PDF or DOCX)
GET	/api/v1/resume/{id}	Get parsed resume data and structured JSON
POST	/api/v1/analysis/start	Trigger full async AI analysis pipeline
GET	/api/v1/analysis/{id}	Get analysis results — poll or listen via WebSocket
POST	/api/v1/jd/adapt	Generate JD-tailored resume variant
POST	/api/v1/analysis/compare	A/B test two resume versions head-to-head
GET	/api/v1/career/trajectory	Get career path prediction + skill gap map
GET	/api/v1/benchmark/{role}	Get industry benchmark data for a role
POST	/api/v1/interview/predict	Generate predicted interview questions
POST	/api/v1/bulk/analyze	Batch analyze multiple resumes (Team plan)

Database Schema

users	id UUID PK, email TEXT UNIQUE, clerk_id TEXT, plan TEXT, created_at
resumes	id UUID PK, user_id FK, filename, s3_key, raw_text, parsed_json JSONB
analyses	id UUID PK, resume_id FK, jd_text, status, results_json JSONB, created_at, completed_at
benchmarks	id UUID PK, role, industry, embedding vector(1536), score FLOAT
jd_matches	id UUID PK, resume_id FK, jd_text, adapted_resume_json JSONB, match_score FLOAT
interview_questions	id UUID PK, analysis_id FK, questions JSONB, created_at
subscriptions	id UUID PK, user_id FK, stripe_id TEXT, plan, status, current_period_end, created_at

Pricing & Monetization

Free	\$0 / month	3 analyses/month, ATS score, section grades, 5 bullet rewrites. Persona/JD adapter locked.
Pro	\$19 / month	Unlimited analyses, all persona views, JD adapter (10/mo), interview predictor, A/B tester, tone + skill checker.
Career	\$39 / month	Everything in Pro + career trajectory AI, salary calibration, GitHub integration, benchmark vs top 10%, multilingual adapter.
Team / B2B	\$199 / month	Up to 20 seats, bulk resume screening, recruiter dashboard, API access, white-label option, priority support.

Revenue projections — month 12 target

Free -> Pro conversion	8–12% (industry average for SaaS tools)
Target MRR (month 12)	\$25,000+ from ~1,000 Pro + 50 Team users
LTV estimate (Pro)	~\$76 average — based on 4-month job search cycle
Additional revenue	API licensing to bootcamps and staffing agencies
Virality hook	Shareable score cards: My resume scored 87/100 on ResumeAI — LinkedIn share

SEO moat	Role-specific landing pages: resume tips for software engineers 2026 etc.
----------	---

Getting Started

Prerequisites

Node.js	v18.17.0 or higher
Python	3.11 or higher
Docker	Docker + Docker Compose v2+
API Keys	Anthropic Claude, Stripe, Clerk, AWS, Pinecone

Quick start (Docker Compose — recommended)

```
git clone https://github.com/yourusername/resume-analyzer.git
cd resume-analyzer
cp .env.example .env # fill in your API keys
docker-compose up --build
```

Services start at: Frontend localhost:3000 | Backend localhost:8000 | Swagger docs localhost:8000/docs

Day-by-day startup guide

Day 1	Monorepo skeleton, Docker Compose, Next.js + FastAPI both return Hello World
Day 2–3	Clerk auth, S3 resume upload, PDF + DOCX parsers returning structured JSON
Day 4–5	First Claude API call — ATS scorer returning JSON score. Store prompts as versioned templates.
Day 6–7	Celery async pipeline. User uploads -> job ID returned immediately -> poll for results.
Week 2	Tone analyzer, persona analyzer, bullet rewriter, interview predictor — one per day.
Week 3	Result page UI, inline bullet rewriter, persona tabs, user dashboard.
Week 4	Stripe billing, feature gating, landing page, deploy to production. Get 10 real users.

Most important rule: build the AI prompts before the UI. Get Claude returning clean, structured JSON first, then build the UI around it. Every analyzer should return a Pydantic schema, not free-form text.